

Computing the Square Root of Two



PLATForm

Programming Languages, Automata Theory,
and Formal Methods

We cannot compute with real numbers.

```
import math  
  
math.sqrt(2) * math.sqrt(2) == 2
```

Python `float` stores the nearest IEEE 754 double-precision approximation:

$$2 \approx 1.4142135623730951$$

We cannot compute with real numbers.

What is this?

```
import math
```

```
math.sqrt(2) * math.sqrt(2) == 2
```

Python `float` stores the nearest IEEE 754 double-precision approximation:

$2 \approx 1.4142135623730951$

The number $\sqrt{2}$ can be defined as:

$$\sup\{x \in \mathbb{Q}^+ : x^2 \leq 2\}$$

Exercise: Prove that $\sqrt{2}$ is not rational.

Theorem: Any real number can be approximated by a rational number up to arbitrary precision.

1;24,51,10

$$\sqrt{2} \approx 1 + \frac{24}{60} + \frac{51}{60^2} + \frac{10}{60^3}$$

Correct up to 5 decimal places.



Babylon, ca. 1800-1600 BCE

The measure is to be increased by its third and this [third] again by its own fourth less the thirty-fourth part [of that fourth]; this is [the value of] the diagonal of a square [whose side is the measure].

$$\sqrt{2} \approx 1 + \frac{1}{3} + \frac{1}{3 \cdot 4} - \frac{1}{3 \cdot 4 \cdot 34}$$

Correct up to 5 decimal places.

Baudhayana Sulba Sutra, ca. 800 BCE

Approximating $\sqrt{2}$ as a rational.

Given $\varepsilon > 0$, find integers p, q such that:

$$\left| \frac{p}{q} - \sqrt{2} \right| < \varepsilon$$

Theorem: Any real number can be approximated by a rational number up to arbitrary precision.

Method 1: Binary Search

Consider $f(x) = x^2 - 2$ on $[1,2]$.

$$f(1) = -1 \quad f(2) = 2.$$

Therefore, for some point $p \in (1,2)$, we have $f(p) = 0$.

Method 1: Binary Search

Consider $f(x) = x^2 - 2$ on $[1,2]$.

$$f(1) = -1 \quad f(2) = 2.$$

Therefore, for some point $p \in (1,2)$, we have $f(p) = 0$.

Clearly, this p is $\sqrt{2}$.

Method 1: Binary Search

Invariant: Maintain an interval (a_k, b_k) such that:

$$f(a_k) < 0 \quad f(b_k) \geq 0.$$

$$\text{Termination: } |b_k - a_k| < \varepsilon.$$

Method 1: Binary Search

Invariant: Maintain an interval (a_k, b_k) such that:

$$f(a_k) < 0 \quad f(b_k) \geq 0.$$

$$\text{Termination: } |b_k - a_k| < \varepsilon.$$

Update?

Method 1: Binary Search

```
a := 1
b := 2
while (b - a > epsilon) do
  m := (a + b) / 2
  if m*m < 2 then a := m else b := m
return a
```

Correct by
construction!

Method 1: Binary Search

```
a := 1
b := 2
while (b - a > epsilon) do
    m := (a + b) / 2
    if m*m < 2 then a := m else b := m
return a
```

Termination?

Method 1: Binary Search

Termination:

The width $b - a$ is halved by each iteration.

Initially, it is 1.

After j iterations, it will be 2^{-j} .

For $2^{-j} < \varepsilon$, we need $j = \lceil \log_2(1/\varepsilon) \rceil$.

Method 1: Binary Search

Termination:

The width $b - a$ is halved by each iteration.

Initially, it is 1.

After j iterations, it will be 2^{-j} .

For $2^{-j} < \varepsilon$, we need $j = \lceil \log_2(1/\varepsilon) \rceil$.

Roughly one digit improvement per three iterations.

Method 1: Binary Search

Termination:

The width $b - a$ is halved by each iteration.

Initially, it is 1.

After j iterations, it will be 2^{-j} .

For $2^{-j} < \varepsilon$, we need $j = \lceil \log_2(1/\varepsilon) \rceil$.

Linear
Convergence

Roughly one digit improvement per three iterations.

Method 1: Binary Search

```
a := 1
b := 2
while (b - a > epsilon) do
    m := (a + b) / 2
    if m*m < 2 then a := m else b := m
return a
```

What is the data structure for a , b , and m ?

What is the cost model for complexity analysis?

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p , q , r , and s as arbitrary-precision integers.

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Query Complexity Model: Suppose computing $f(x)$
takes $O(1)$ time and all operations take no time.

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Query Complexity Model: Suppose computing $f(x)$
takes $O(1)$ time and all operations take no time.

Then this algorithm runs in $\Theta(\log(1/\varepsilon))$.

Optimal!

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Bit-Operation Model: Suppose each
bit-operation takes $O(1)$ time.

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p , q , r , and s as arbitrary-precision integers.

$$m = \frac{a + b}{2} = \frac{ps + rq}{2qs}.$$

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

$$m = \frac{a + b}{2} = \frac{ps + rq}{2qs}.$$

Let u_k and v_k be the bit size of the denominators of a_k and b_k . Then:

$$(u, v) \longmapsto (u + v + 1, v) \quad \text{or} \quad (u, u + v + 1).$$

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

$$m = \frac{a + b}{2} = \frac{ps + rq}{2qs}.$$

Let u_k and v_k be the bit size of the denominators of a_k and b_k . Then:

$$(u, v) \longmapsto (u + v + 1, v) \quad \text{or} \quad (u, u + v + 1).$$

The branches give denominators of size 1, 2, 4, 6, 11, 18, 30, 49, ...

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

$$m = \frac{a + b}{2} = \frac{ps + rq}{2qs}.$$

Let u_k and v_k be the bit size of the denominators of a_k and b_k . Then:

$$(u, v) \longmapsto (u + v + 1, v) \quad \text{or} \quad (u, u + v + 1).$$

This is upper-bounded by the recurrence $s_{k+1} = s_k + s_{k-1} + O(1)$

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

$$m = \frac{a + b}{2} = \frac{ps + rq}{2qs}.$$

Let u_k and v_k be the bit size of the denominators of a_k and b_k . Then:

$$(u, v) \longmapsto (u + v + 1, v) \quad \text{or} \quad (u, u + v + 1).$$

Hence, cost of n -bits: $M(\Theta(\varphi^n))$.

integer
multiplication
complexity

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Hence, cost of approximating up to n -bits: $\widetilde{\Theta}(\varphi^n)$

Assuming the
fastest algorithm
for multiplication.

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Hence, cost of approximating up to n -bits: $\widetilde{\Theta}(\varphi^n)$ Assuming the fastest algorithm for multiplication.

Exact rational arithmetic applied naively to an iterative loop is exponential.

This is called *intermediate expression swell*.

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p , q , r , and s as arbitrary-precision integers.

Option A: reduce every step.

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Option A: reduce every step.

gcd of k -bit integers costs $O(M(k)\log k)$.

It still does not reduce the denominators in the worst case.

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Option B: Observe that:

$$b_k - a_k = 2^{-k}$$

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Option B: Observe that:

$$b_k - a_k = 2^{-k}$$

$$a_k = \frac{A_k}{2^k} \quad b_k = \frac{A_k + 1}{2^k} \quad A_k \in \mathbb{Z}$$

Method 1: Binary Search

Endpoints are exact rationals $a = p/q$ and $b = r/s$,
with p, q, r , and s as arbitrary-precision integers.

Option B: Observe that:

$$b_k - a_k = 2^{-k}$$

$$a_k = \frac{A_k}{2^k} \quad b_k = \frac{A_k + 1}{2^k} \quad A_k \in \mathbb{Z}$$

The entire
state is a
 $k + 1$ bit
integer.

Method 2: Binary Search with Dyadic Numbers

```
A := 1 ; k := 0
while 2(-k) > epsilon do
    if (2*A + 1)2 < 2(2k+3)
    then A := 2*A + 1
    else A := 2*A
    k := k + 1
return A / 2k
```

Method 2: Binary Search with Dyadic Numbers

```
n := ceil(log2(1/epsilon))  
A := 1  
for k := 0 to n-1 do  
    if (2*A + 1)^2 < 2^(2k+3)  
        then A := 2*A + 1  
        else A := 2*A  
return A / 2^n
```

Method 2: Binary Search with Dyadic Numbers

Let $\varepsilon = P/Q$ and $n = \lceil \log_2(1/\varepsilon) \rceil$.

$$\Theta(\log(P) + \log(Q)) + \sum_{k=0}^{n-1} (M(k) + \Theta(k)) = \Theta(nM(n))$$

Method 2: Binary Search with Dyadic Numbers

Let $\varepsilon = P/Q$ and $n = \lceil \log_2(1/\varepsilon) \rceil$.

$$\Theta(\log(P) + \log(Q)) + \sum_{k=0}^{n-1} (M(k) + \Theta(k)) = \Theta(nM(n))$$

Down from exponential: $\widetilde{\Theta}(\varphi^n) \rightarrow \widetilde{\Theta}(n^2)$

Method 2: Binary Search with Dyadic Numbers

$$\sqrt{2} = 1.0110101000001001111\dots$$

The branch sequence of dyadic bisection is eventually periodic $\iff$ the target is rational.

And therefore we cannot predict $(2^k A + 1)^2 < 2^{2k+3}$.

Method 3: Binary Search by Mediant

We change the *midpoint*:

Observe that if $\frac{p}{q} < \frac{p'}{q'}$, then: $\frac{p}{q} < \frac{p + p'}{q + q'} < \frac{p'}{q'}$

And we can use this
for the binary search!

Method 3: Binary Search by Mediant

```
(a_n, a_d) := (1, 1)
(b_n, b_d) := (2, 1)
while (b - a) > epsilon do
    (m_n, m_d) := (a_n + b_n, a_d + b_d)
    if m_n*m_n < 2*m_d*m_d
        then (a_n, a_d) := (m_n, m_d)
        else (b_n, b_d) := (m_n, m_d)
return a_n / a_d
```

Method 3: Binary Search by Mediant

```
(a_n, a_d) := (1, 1)
(b_n, b_d) := (2, 1)
while (b - a) > epsilon do
    (m_n, m_d) := (a_n + b_n, a_d + b_d)
    if m_n*m_n < 2*m_d*m_d
    then (a_n, a_d) := (m_n, m_d)
    else (b_n, b_d) := (m_n, m_d)
return a_n / a_d
```

Method 3: Binary Search by Mediant

```
(b - a) > epsilon
```

$$\frac{b_n}{b_d} - \frac{a_n}{a_d} = \frac{a_d b_n - b_d a_n}{b_d a_d}$$

Method 3: Binary Search by Mediant

```
(b - a) > epsilon
```

Let $\delta_n = a_d b_n - b_d a_n$

$$\frac{b_n}{b_d} - \frac{a_n}{a_d} = \frac{a_d b_n - b_d a_n}{b_d a_d}$$

Method 3: Binary Search by Mediant

$$(b - a) > \text{epsilon}$$

$$\frac{b_n}{b_d} - \frac{a_n}{a_d} = \frac{a_d b_n - b_d a_n}{b_d a_d}$$

$$\text{Let } \delta_n = a_d b_n - b_d a_n$$

$$\text{Clearly, } \delta_1 = 1$$

Method 3: Binary Search by Mediant

$$(b - a) > \text{epsilon}$$

$$\text{Suppose } b' = m = \frac{a_n + b_n}{a_d + b_d}$$

$$\text{Let } \delta_n = a_d b_n - b_d a_n$$

$$\text{Clearly, } \delta_1 = 1$$

Method 3: Binary Search by Mediant

$$(b - a) > \text{epsilon}$$

$$\text{Suppose } b' = m = \frac{a_n + b_n}{a_d + b_d}$$

$$\text{Let } \delta_n = a_d b_n - b_d a_n$$

$$\text{Clearly, } \delta_1 = 1$$

$$\delta_{n+1} = a_d b'_n - b'_d a_n$$

Method 3: Binary Search by Mediant

$$(b - a) > \text{epsilon}$$

$$\text{Suppose } b' = m = \frac{a_n + b_n}{a_d + b_d}$$

$$\text{Let } \delta_n = a_d b_n - b_d a_n$$

$$\text{Clearly, } \delta_1 = 1$$

$$\delta_{n+1} = a_d(a_n + b_n) - (a_d + b_d)a_n$$

Method 3: Binary Search by Mediant

$$(b - a) > \text{epsilon}$$

$$\text{Suppose } b' = m = \frac{a_n + b_n}{a_d + b_d}$$

$$\text{Let } \delta_n = a_d b_n - b_d a_n$$

$$\text{Clearly, } \delta_1 = 1$$

$$\delta_{n+1} = a_d b_n - b_d a_n$$

Method 3: Binary Search by Mediant

$$(b - a) > \text{epsilon}$$

$$\text{Suppose } b' = m = \frac{a_n + b_n}{a_d + b_d}$$

$$\text{Let } \delta_n = a_d b_n - b_d a_n$$

$$\text{Clearly, } \delta_1 = 1$$

$$\delta_{n+1} = a_d b_n - b_d a_n = 1$$

Method 3: Binary Search by Mediant

```
(b - a) > epsilon
```

Let $\delta_n = a_d b_n - b_d a_n$

Clearly, $\delta_1 = 1$

$$\delta_{n+1} = a_d b_n - b_d a_n = 1$$

$$\text{Therefore: } b - a = \frac{1}{a_d b_d}$$

Method 3: Binary Search by Mediant

```
(a_n, a_d) := (1, 1)
(b_n, b_d) := (2, 1)
while Q > P * a_d * b_d do
    (m_n, m_d) := (a_n + b_n, a_d + b_d)
    if m_n*m_n < 2*m_d*m_d
    then (a_n, a_d) := (m_n, m_d)
    else (b_n, b_d) := (m_n, m_d)
return a_n / a_d
```

$$\varepsilon = \frac{P}{Q}$$

Method 3: Binary Search by Mediant

step	mediant	branch
1	$\frac{3}{2}$	L
2	$\frac{4}{3}$	R
3	$\frac{7}{5}$	R
4	$\frac{10}{7}$	L
5	$\frac{17}{12}$	L
6	$\frac{24}{17}$	R
7	$\frac{41}{29}$	R
8	$\frac{58}{41}$	L
9	$\frac{99}{70}$	L
10	$\frac{140}{99}$	R
11	$\frac{239}{169}$	R

Method 3: Binary Search by Mediant

step	mediant	branch
1	3/2	L
2	4/3	R
3	7/5	R
4	10/7	L
5	17/12	L
6	24/17	R
7	41/29	R
8	58/41	L
9	99/70	L
10	140/99	R
11	239/169	R

$(LRRL)^{\omega}$

Why?

Method 3: Binary Search by Mediant

$$B = \begin{pmatrix} b_n & a_n \\ b_d & a_d \end{pmatrix}, \quad a = \frac{a_n}{a_d} < \sqrt{2} < \frac{b_n}{b_d} = b .$$

Method 3: Binary Search by Mediant

k	B_k	a	b	mediant	test	branch
0	$\begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$	$\frac{1}{1}$	$\frac{2}{1}$	$\frac{3}{2}$	$9 > 8$	L
1	$\begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}$	$\frac{1}{1}$	$\frac{3}{2}$	$\frac{4}{3}$	$16 < 18$	R
2	$\begin{pmatrix} 3 & 4 \\ 2 & 3 \end{pmatrix}$	$\frac{4}{3}$	$\frac{3}{2}$	$\frac{7}{5}$	$49 < 50$	R
3	$\begin{pmatrix} 3 & 7 \\ 2 & 5 \end{pmatrix}$	$\frac{7}{5}$	$\frac{3}{2}$	$\frac{10}{7}$	$100 > 98$	L
4	$\begin{pmatrix} 10 & 7 \\ 7 & 5 \end{pmatrix}$	$\frac{7}{5}$	$\frac{10}{7}$			

Method 3: Binary Search by Mediant

$$B = \begin{pmatrix} b_n & a_n \\ b_d & a_d \end{pmatrix}, \quad a = \frac{a_n}{a_d} < \sqrt{2} < \frac{b_n}{b_d} = b.$$

$$H = B_4 B_0^{-1} = \begin{pmatrix} 10 & 7 \\ 7 & 5 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 2 & 3 \end{pmatrix}$$

Method 3: Binary Search by Mediant

$$B = \begin{pmatrix} b_n & a_n \\ b_d & a_d \end{pmatrix}, \quad a = \frac{a_n}{a_d} < \sqrt{2} < \frac{b_n}{b_d} = b.$$

$$H = B_4 B_0^{-1} = \begin{pmatrix} 10 & 7 \\ 7 & 5 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 2 & 3 \end{pmatrix}$$

$$B_{4k} = H^k B_0$$

Proof by induction!

Method 4: Iterative Refinement

```
(a_n, a_d) := (1, 1)
(b_n, b_d) := (2, 1)
while Q > P * a_d * b_d do
    (a_n, a_d) := (3*a_n + 4*a_d, 2*a_n + 3*a_d)
    (b_n, b_d) := (3*b_n + 4*b_d, 2*b_n + 3*b_d)
return a_n / a_d
```

Method 4: Iterative Refinement

Step	a	b	a_d.b_d
0	1/1	2/1	1
1	7/5	10/7	35
2	41/29	58/41	1189
3	239/169	338/239	40391

Observe that $b = \frac{2}{a}$.

Method 4.2: Iterative Refinement II

```
(p, q) := (1, 1)
while Q > P * p * q do
    (p, q) := (3*p + 4*q, 2*p + 3*q)
return p / q
```

Method 4.2: Iterative Refinement II

```
(p, q) := (1, 1)
while Q > P * p * q do
    (p, q) := (3*p + 4*q, 2*p + 3*q)
return p / q
```

Convergence?

Method 4.2: Iterative Refinement II

$$(p, q) \mapsto (3p + 4q, 2p + 3q)$$

Method 4.2: Iterative Refinement II

$$(p, q) \mapsto (3p + 4q, 2p + 3q)$$

Let (p, q) denote a member of $\mathbb{Z} \left[\sqrt{2} \right], p + q\sqrt{2}$.

Then the transformation is:

$$p + q\sqrt{2} \mapsto (3p + 4q) + (2p + 3q)\sqrt{2}$$

Method 4.2: Iterative Refinement II

$$(p, q) \mapsto (3p + 4q, 2p + 3q)$$

Let (p, q) denote a member of $\mathbb{Z}[\sqrt{2}]$, $p + q\sqrt{2}$.

Then the transformation is:

$$p + q\sqrt{2} \mapsto (3p + 4q) + (2p + 3q)\sqrt{2} = (3 + 2\sqrt{2})(p + q\sqrt{2})$$

Method 4.2: Iterative Refinement II

$$(p, q) \mapsto (3p + 4q, 2p + 3q)$$

Let (p, q) denote a member of $\mathbb{Z}[\sqrt{2}]$, $p + q\sqrt{2}$.

Then the transformation is:

$$p + q\sqrt{2} \mapsto (3p + 4q) + (2p + 3q)\sqrt{2} = \left(1 + \sqrt{2}\right)^2 \left(p + q\sqrt{2}\right)$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

$$p + q\sqrt{2} \mapsto (3p + 4q) + (2p + 3q)\sqrt{2} = \left(1 + \sqrt{2}\right)^2 \left(p + q\sqrt{2}\right)$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Therefore, the transformation is:

$$p + q\sqrt{2} \mapsto \tau^2 (p + q\sqrt{2})$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Therefore, the transformation is:

$$p + q\sqrt{2} \mapsto \tau^2 (p + q\sqrt{2})$$

$$\text{Also } p_0 + q_0\sqrt{2} = 1 + \sqrt{2} = \tau.$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Which gives us:

$$p_k + q_k\sqrt{2} = \tau (\tau^2)^k = \tau^{2k+1}$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Which gives us:

$$p_k + q_k\sqrt{2} = \tau (\tau^2)^k = \tau^{2k+1}$$

$$\overline{p_k + q_k\sqrt{2}} = p_k - q_k\sqrt{2}$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Which gives us:

$$p_k + q_k\sqrt{2} = \tau (\tau^2)^k = \tau^{2k+1}$$

$$\overline{p_k + q_k\sqrt{2}} = p_k - q_k\sqrt{2} = \overline{\tau^{2k+1}} = \bar{\tau}^{2k+1}$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Which gives us:

$$p_k = \frac{\tau^{2k+1} + \bar{\tau}^{2k+1}}{2}$$

$$q_k = \frac{\tau^{2k+1} - \bar{\tau}^{2k+1}}{2\sqrt{2}}$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Which gives us:

$$p_k = \frac{\tau^{2k+1} + \bar{\tau}^{2k+1}}{2} \qquad q_k = \frac{\tau^{2k+1} - \bar{\tau}^{2k+1}}{2\sqrt{2}}$$

$$\text{Also } \tau \cdot \bar{\tau} = \left(1 + \sqrt{2}\right) \left(1 - \sqrt{2}\right) = -1$$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Which gives us:

$$p_k = \frac{\tau^{2k+1} + \bar{\tau}^{2k+1}}{2} \qquad q_k = \frac{\tau^{2k+1} - \bar{\tau}^{2k+1}}{2\sqrt{2}}$$

That is $\bar{\tau} = -\tau^{-1}$

Method 4.2: Iterative Refinement II

Observe that $\tau = 1 + \sqrt{2}$ is the fundamental unit of $\mathbb{Z}[\sqrt{2}]$!

Which gives us:

$$p_k = \frac{\tau^{2k+1} + (-\tau)^{-2k-1}}{2}$$

$$q_k = \frac{\tau^{2k+1} - (-\tau)^{-2k-1}}{2\sqrt{2}}$$

Method 4.2: Iterative Refinement II

```
(p, q) := (1, 1)
while Q > P * p * q do
    (p, q) := (3*p + 4*q, 2*p + 3*q)
return p / q
```

The loop terminates when $p_k \cdot q_k \geq \frac{1}{\varepsilon}$.

Method 4.2: Iterative Refinement II

Which gives us:

$$p_k \cdot q_k = \frac{\tau^{2k+1} + (-\tau)^{-2k-1}}{2} \cdot \frac{\tau^{2k+1} - (-\tau)^{-2k-1}}{2\sqrt{2}} = \frac{\tau^{4k+2} - (-\tau)^{-4k-2}}{4\sqrt{2}}$$

Method 4.2: Iterative Refinement II

Which gives us:

$$p_k \cdot q_k = \frac{\tau^{2k+1} + (-\tau)^{-2k-1}}{2} \cdot \frac{\tau^{2k+1} - (-\tau)^{-2k-1}}{2\sqrt{2}} = \frac{\tau^{4k+2} - \tau^{-4k-2}}{4\sqrt{2}}$$
$$\approx \frac{\tau^{4k+2}}{4\sqrt{2}}$$

Method 4.2: Iterative Refinement II

So we want:

$$\frac{\tau^{4k+2}}{4\sqrt{2}} \geq \frac{1}{\varepsilon}$$

Method 4.2: Iterative Refinement II

So we want:

$$(4k + 2)\log_2 \tau - \log_2(4\sqrt{2}) \geq \log_2(1/\varepsilon)$$

Method 4.2: Iterative Refinement II

So we want:

$$k \geq \frac{\log_2(1/\varepsilon) + \log_2(4\sqrt{2})}{4 \log_2 \tau} - \frac{1}{2}$$

Method 4.2: Iterative Refinement II

So we want:

$$n = \left\lceil \frac{\log_2(1/\varepsilon) + \log_2(4\sqrt{2})}{4 \log_2 \tau} - \frac{1}{2} \right\rceil$$

Method 4.2: Iterative Refinement II

Approximately:

$$n \approx 0.19661 \log_2(1/\varepsilon)$$

Method 4.2: Iterative Refinement II

Approximately:

$$n \approx 0.19661 \log_2(1/\varepsilon)$$

Convergence is $O(\log(1/\varepsilon))$.

Each step multiplies $p_k q_k$ by a constant τ^4 .

Method 4.2: Iterative Refinement II

```
n = ceil(0.2*log2(1/epsilon))  
(p, q) := (1, 1)  
for k := 0 to n-1 do  
    (p, q) := (3*p + 4*q, 2*p + 3*q)  
return p / q
```

Method 4.2: Iterative Refinement II

What is this algorithm?

```
n = ceil(0.2*log2(1/epsilon))  
(p, q) := (1, 1)  
for k := 0 to n-1 do  
    (p, q) := (3*p + 4*q, 2*p + 3*q)  
return p / q
```

Method 4.2: Iterative Refinement II

What is this algorithm?

$$\text{Let } n = \lceil 0.2 \log_2(1/\varepsilon) \rceil$$

Method 4.2: Iterative Refinement II

What is this algorithm?

Let $n = \lceil 0.2 \log_2(1/\varepsilon) \rceil$

Compute $p_n, q_n \in \mathbb{Z}$ such that $p_n + q_n\sqrt{n} = \left(1 + \sqrt{2}\right)^{2n+1}$.

Method 4.2: Iterative Refinement II

What is this algorithm?

Let $n = \lceil 0.2 \log_2(1/\varepsilon) \rceil$

Compute $p_n, q_n \in \mathbb{Z}$ such that $p_n + q_n\sqrt{n} = \left(1 + \sqrt{2}\right)^{2n+1}$.

Return $\frac{p_n}{q_n}$.

Method 4.2: Iterative Refinement II

What is this algorithm?

Let $n = \lceil 0.2 \log_2(1/\varepsilon) \rceil$

Compute $p_n, q_n \in \mathbb{Z}$ such that $p_n + q_n\sqrt{n} = \left(1 + \sqrt{2}\right)^{2n+1}$.

Return $\frac{p_n}{q_n}$.

Why should this take
linear time in n ?

Method 5: Refining with Fast Power

$$\tau \rightarrow \tau^3 \rightarrow \tau^5 \rightarrow \tau^7 \rightarrow \dots$$

$$\tau \rightarrow \tau^2 \rightarrow \tau^4 \rightarrow \tau^8 \rightarrow \dots$$

Square at each step.

Method 5: Refining with Fast Power

$$\tau \rightarrow \tau^3 \rightarrow \tau^5 \rightarrow \tau^7 \rightarrow \dots$$

$$\tau \rightarrow \tau^2 \rightarrow \tau^4 \rightarrow \tau^8 \rightarrow \dots \quad \text{Square at each step.}$$

We will end up computing τ^{2^k} in k squarings.

Method 5: Refining with Fast Power

$$\tau \rightarrow \tau^3 \rightarrow \tau^5 \rightarrow \tau^7 \rightarrow \dots$$

$$\tau \rightarrow \tau^2 \rightarrow \tau^4 \rightarrow \tau^8 \rightarrow \dots$$

Square at each step.

For what k is $2^k > 2n + 1$?

Method 5: Refining with Fast Power

$$\tau \rightarrow \tau^3 \rightarrow \tau^5 \rightarrow \tau^7 \rightarrow \dots$$

$$\tau \rightarrow \tau^2 \rightarrow \tau^4 \rightarrow \tau^8 \rightarrow \dots$$

Square at each step.

Therefore, take $k = \lceil \log_2(2n + 1) \rceil$.

Method 5: Refining with Fast Power

$$\tau \rightarrow \tau^3 \rightarrow \tau^5 \rightarrow \tau^7 \rightarrow \dots$$

$$\tau \rightarrow \tau^2 \rightarrow \tau^4 \rightarrow \tau^8 \rightarrow \dots \quad \text{Square at each step.}$$

Therefore, take $k = \lceil \log_2 (0.4 \log_2(1/\varepsilon) + 0.6) \rceil$.

Method 5: Refining with Fast Power

Therefore, take $k = \lceil \log_2 (0.4 \log_2(1/\varepsilon) + 0.6) \rceil$.

What is the squaring step?

Method 5: Refining with Fast Power

Therefore, take $k = \lceil \log_2 (0.4 \log_2(1/\varepsilon) + 0.6) \rceil$.

What is the squaring step?

$$p + q\sqrt{2} \mapsto (p + q\sqrt{2}) (p + q\sqrt{2}) = (p^2 + 2q^2) + (2pq) \sqrt{2}$$

Method 5: Refining with Fast Power

Therefore, take $k = \lceil \log_2 (0.4 \log_2(1/\varepsilon) + 0.6) \rceil$.

What is the squaring step?

$$(p, q) \mapsto (p^2 + 2q^2, 2pq)$$

Method 5: Refining with Fast Power

```
n = ceil(log2(0.4*log2(1/epsilon)+0.6))
(p, q) := (1, 1)
for k := 0 to n-1 do
    (p, q) := (p*p + 2*q*q, 2*p*q)
return p / q
```

Method 5: Refining with Fast Power

```
n = ceil(log2(0.4*log2(1/epsilon)+0.6))
(p, q) := (1, 1)
for k := 0 to n-1 do
    (p, q) := (p*p + 2*q*q, 2*p*q)
return p / q
```

This converges quadratically.

Method 5: Refining with Fast Power

This converges quadratically.

Let $m = 2^k$

$$p_k + q_k\sqrt{2} = \tau^m$$

$$p_k - q_k\sqrt{2} = \bar{\tau}^m = \tau^{-m}$$

Method 5: Refining with Fast Power

This converges quadratically.

Let $m = 2^k$

$$p_k + q_k\sqrt{2} = \tau^m$$

$$p_k - q_k\sqrt{2} = \bar{\tau}^m = \tau^{-m}$$

$$q_k = \frac{\tau^m - \tau^{-m}}{2\sqrt{2}}$$

Method 5: Refining with Fast Power

This converges quadratically.

Let $m = 2^k$

$$p_k + q_k\sqrt{2} = \tau^m$$

$$p_k - q_k\sqrt{2} = \bar{\tau}^m = \tau^{-m}$$

$$\left| \frac{p}{q} - \sqrt{2} \right| = \left| \frac{p - q\sqrt{2}}{q} \right| = \frac{\tau^{-m} 2\sqrt{2}}{\tau^m - \tau^{-m}} = \frac{2\sqrt{2}}{\tau^{2m} - 1}$$

Method 5: Refining with Fast Power

This converges quadratically.

Let $m = 2^k$

$$p_k + q_k\sqrt{2} = \tau^m$$

$$p_k - q_k\sqrt{2} = \bar{\tau}^m = \tau^{-m}$$

$$e_k = \frac{2\sqrt{2}}{\tau^{2m} - 1}$$

Method 5: Refining with Fast Power

This converges quadratically.

Let $m = 2^k$

$$p_k + q_k\sqrt{2} = \tau^m$$

$$p_k - q_k\sqrt{2} = \bar{\tau}^m = \tau^{-m}$$

As $k \rightarrow k + 1$,
 m doubles.

$$e : \frac{2\sqrt{2}}{\tau^{2m} - 1} \rightarrow \frac{2\sqrt{2}}{\tau^{4m} - 1}$$

Method 5: Refining with Fast Power

This converges quadratically.

Let $m = 2^k$

$$p_k + q_k\sqrt{2} = \tau^m$$

$$p_k - q_k\sqrt{2} = \bar{\tau}^m = \tau^{-m}$$

$$\frac{e_{k+1}}{e_k^2} = \frac{(\tau^{2m} - 1)^2}{2\sqrt{2}(\tau^{4m} - 1)} = \frac{(\tau^{2m} - 1)^2}{2\sqrt{2}(\tau^{2m} - 1)(\tau^{2m} + 1)} = \frac{(\tau^{2m} - 1)}{2\sqrt{2}(\tau^{2m} + 1)} \approx C$$

Method 5: Refining with Fast Power

This converges quadratically.

Let $m = 2^k$

$$p_k + q_k\sqrt{2} = \tau^m$$

$$p_k - q_k\sqrt{2} = \bar{\tau}^m = \tau^{-m}$$

That is $e_{k+1} \approx C \cdot e_k^2$.

quadratic!

Method 1: Binary Search

familiar



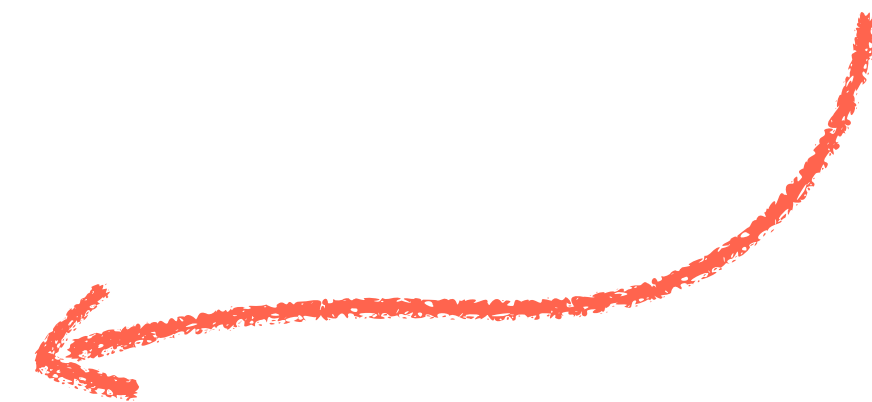
Method 2: Binary Search with Dyadic Numbers

Method 3: Binary Search by Mediant

Method 4: Iterative Refinement

familiar?

Method 5: Refining with Fast Power



Method 5: Refining with Fast Power

$$(p, q) \mapsto (p^2 + 2q^2, 2pq)$$

$$\frac{p}{q} \rightarrow \frac{p^2 + 2q^2}{2pq}$$

Method 5: Refining with Fast Power

$$(p, q) \mapsto (p^2 + 2q^2, 2pq)$$

$$\frac{p}{q} \rightarrow \frac{1}{2} \left(\frac{p}{q} + \frac{2q}{p} \right)$$

Method 5: Refining with Fast Power

$$(p, q) \mapsto (p^2 + 2q^2, 2pq)$$

$$a_k \rightarrow \frac{1}{2} \left(a_k + \frac{2}{a_k} \right)$$

Divide and
Average!

1;24,51,10

$$\sqrt{2} \approx 1 + \frac{24}{60} + \frac{51}{60^2} + \frac{10}{60^3}$$

Correct up to 5 decimal places.



Babylon, ca. 1800-1600 BCE

Initial: 1



Babylon, ca. 1800-1600 BCE

First Approximation:

$$\frac{1}{2} \left(1 + \frac{2}{1} \right) = 1.5 = 1;30$$



Babylon, ca. 1800-1600 BCE

Second Approximation:

$$\frac{1}{2} \left(1.5 + \frac{2}{1.5} \right) = \frac{17}{12}$$

1;25



Babylon, ca. 1800-1600 BCE

Third Approximation:

$$\frac{1}{2} \left(\frac{17}{12} + \frac{2 \times 12}{17} \right) = \frac{577}{408}$$

$1; 24, 51, 10, 35, 17, \dots 60$



Babylon, ca. 1800-1600 BCE

1;24,51,10

$$\sqrt{2} \approx 1 + \frac{24}{60} + \frac{51}{60^2} + \frac{10}{60^3}$$

1;24,51,10,35,17,...60



Babylon, ca. 1800-1600 BCE

The measure is to be increased by its third and this [third] again by its own fourth less the thirty-fourth part [of that fourth]; this is [the value of] the diagonal of a square [whose side is the measure].

$$\sqrt{2} \approx 1 + \frac{1}{3} + \frac{1}{3 \cdot 4} - \frac{1}{3 \cdot 4 \cdot 34}$$

Correct up to 5 decimal places.

Baudhayana Sulba Sutra, ca. 800 BCE

Observe:

$$1 + \frac{1}{3} + \frac{1}{3 \cdot 4} = \frac{17}{12}$$

$$1 + \frac{1}{3} + \frac{1}{3 \cdot 4} - \frac{1}{3 \cdot 4 \cdot 34} = \frac{577}{408}$$

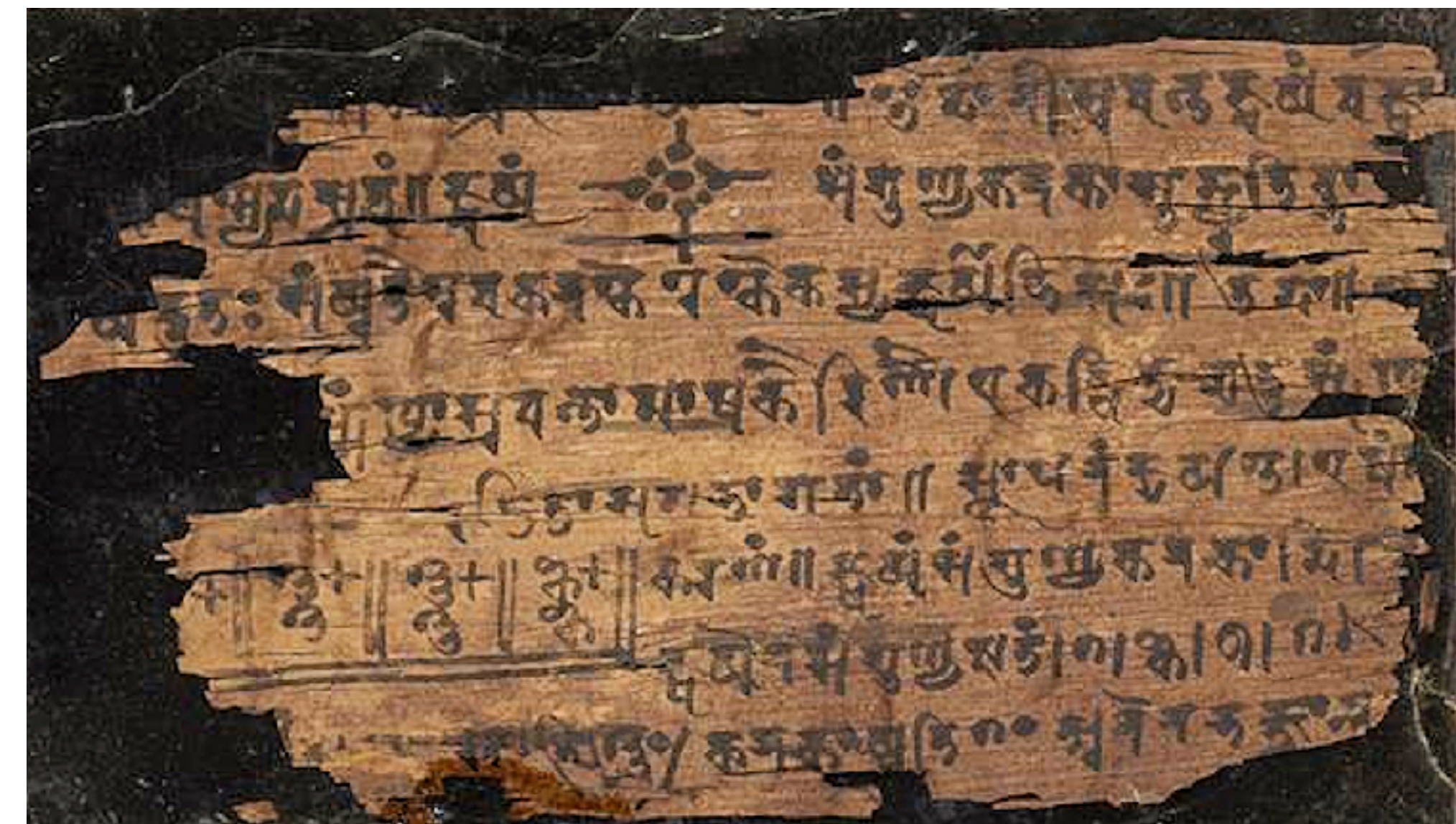
Generalisation:

Consider $N = a^2 + b$.

Start with $a_0 = a$.



Hero of Alexandria



Bakhshali Manuscript

Generalisation:

Consider $N = a^2 + b$.

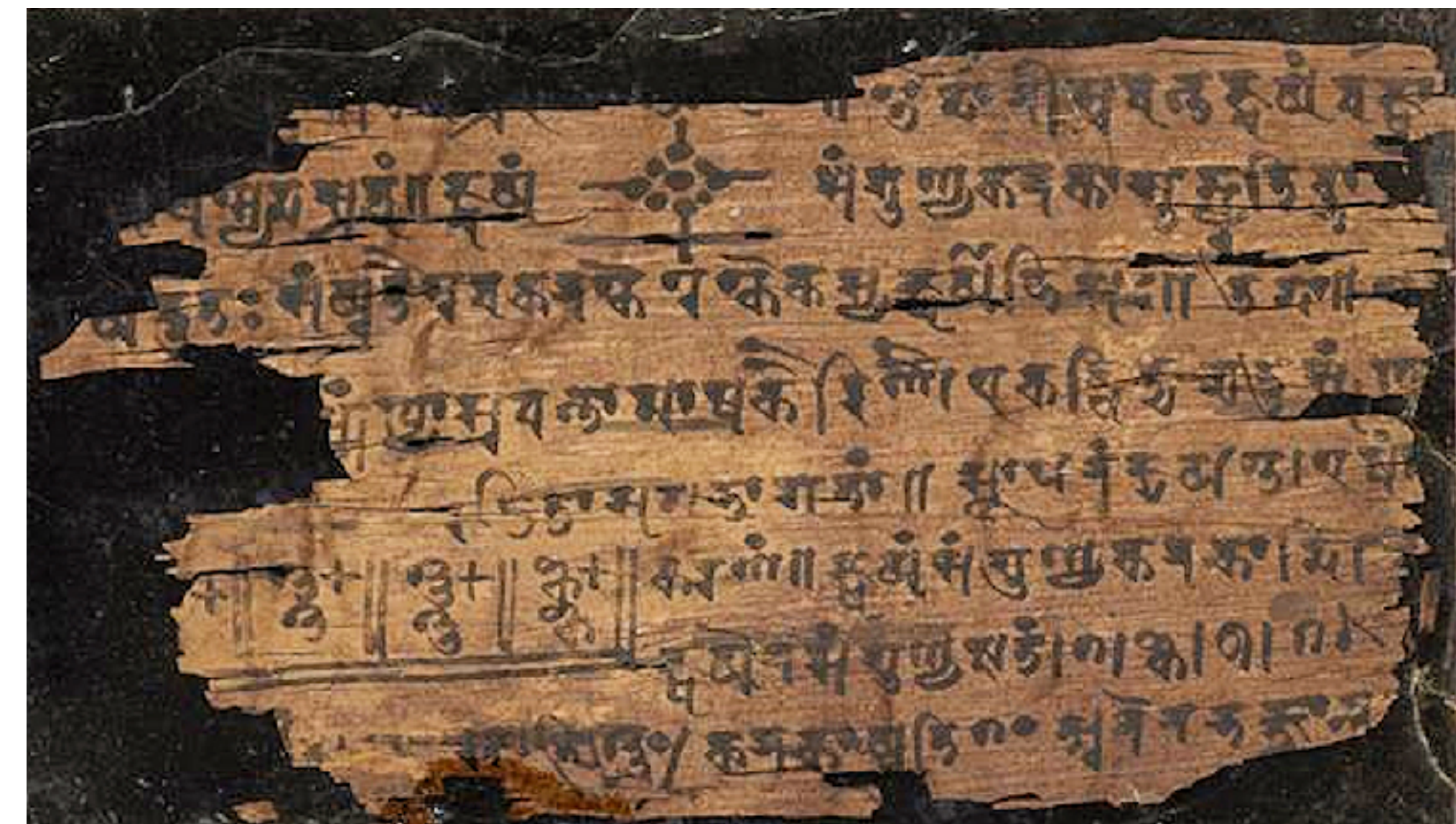
Start with $a_0 = a$.

Then you get $a_1 = a + \frac{b}{2a}$.

$$a_2 = a + \frac{b}{2a} - \frac{(b/2a)^2}{2(a + b/(2a))}.$$



Hero of Alexandria



Bakhshali Manuscript

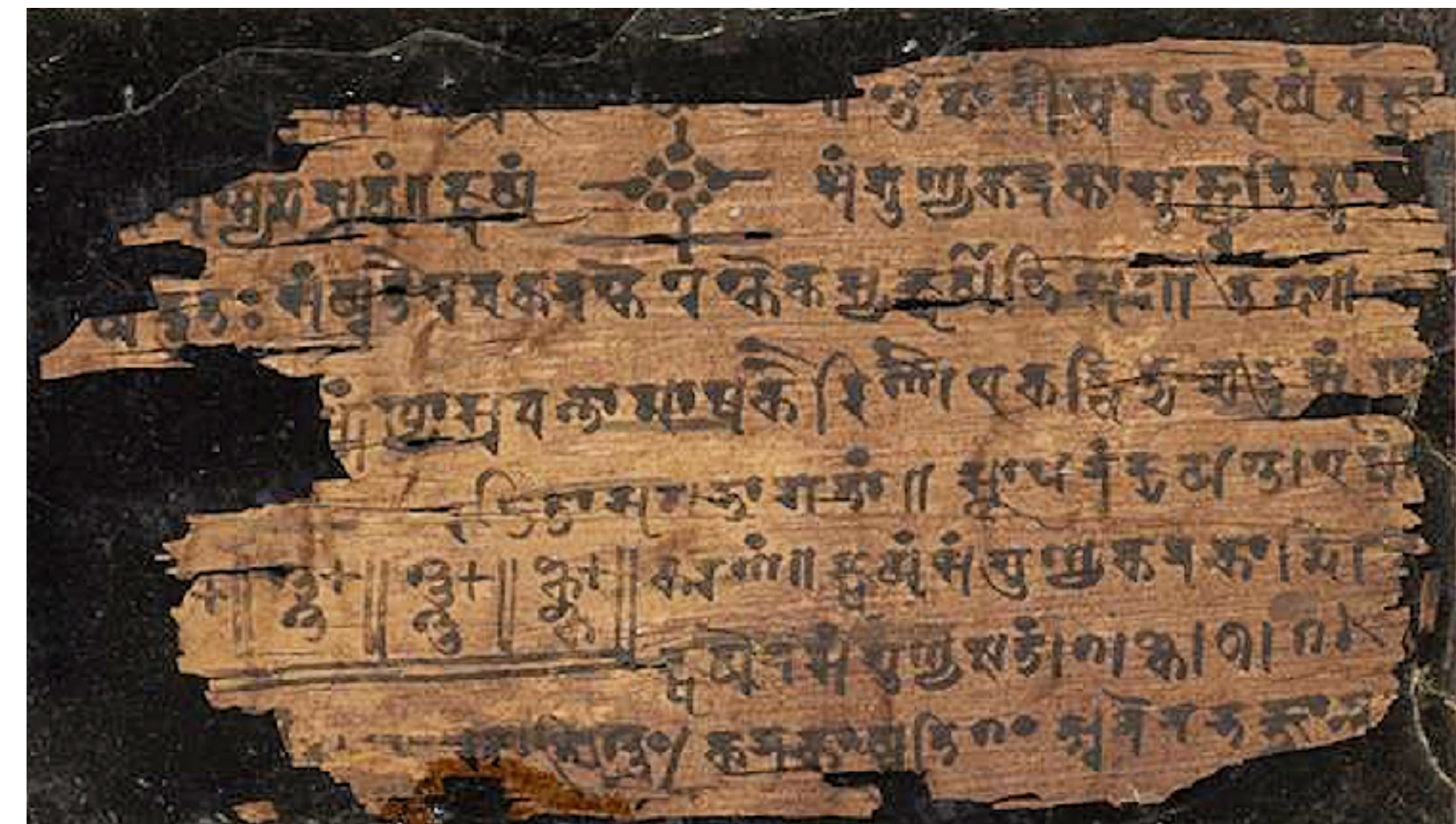
Generalisation:

Computes $\sqrt{720}$.

$$\sqrt{a^2 + b} \approx a + \frac{b}{2a} - \frac{(b/2a)^2}{2(a + b/(2a))}.$$



Hero of Alexandria

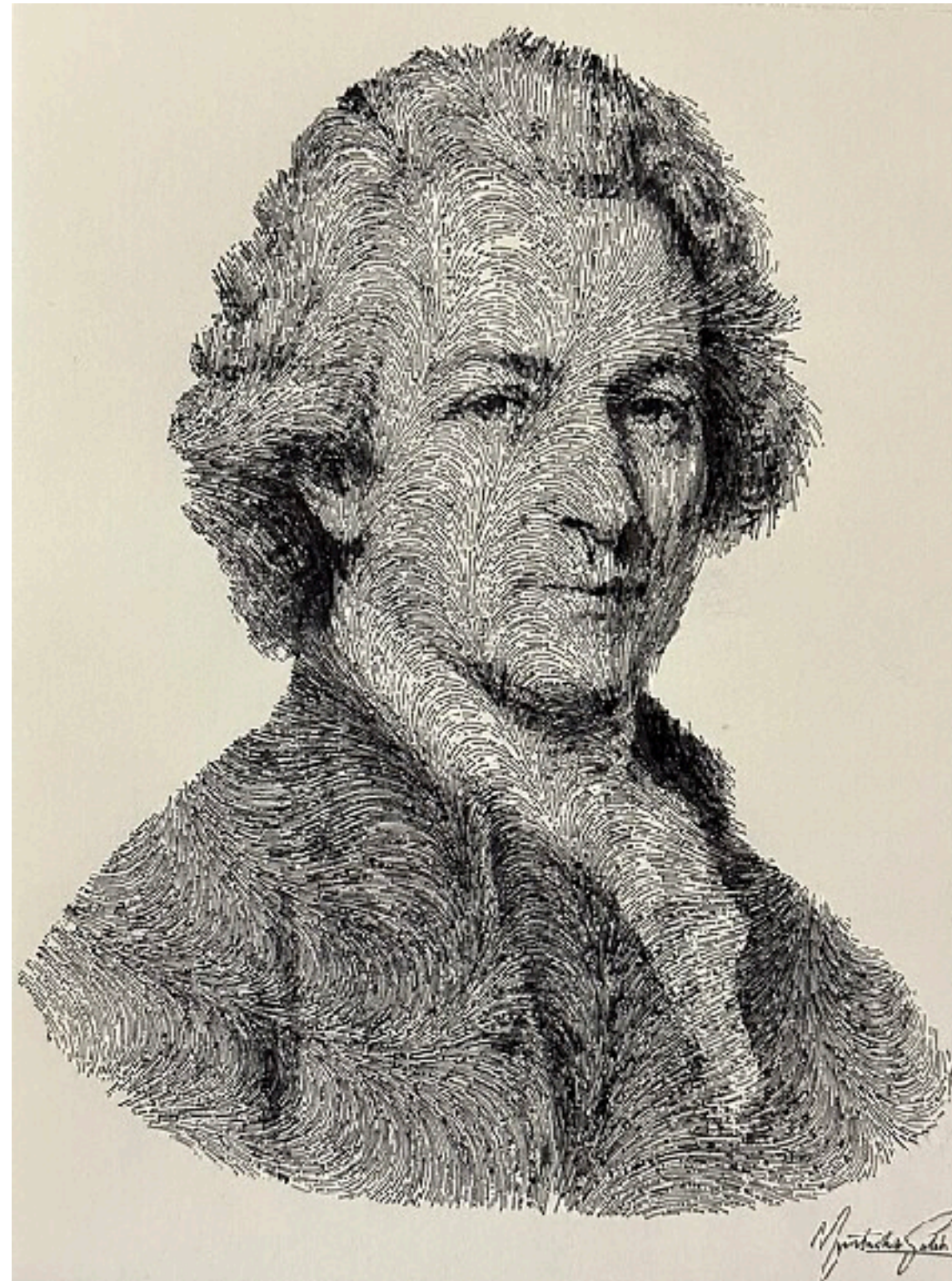


Bakhshali Manuscript

Generalisation:



Isaac Newton



Joseph Raphson



Thomas Simpson

We cannot compute with real numbers.

What is this?

```
import math
```

```
math.sqrt(2) * math.sqrt(2) == 2
```



A version of Method 5, for adaptive-precision floating points with a seed from a look-up table.